

FAANG / Top-Tier

System Design & Machine Coding

Complete Interview Preparation Guide · Hinglish Edition

**17 System Design
Questions**

**9 Machine Coding
Problems**

Extra FAANG Questions

□ Is Guide Ko Kaise Use Karo

Ye guide teen cheezon pe focus karti hai: System Design (broad architecture), Machine Coding (low-level implementation), aur extra FAANG-specific prep. Har question ke saath milega:

- □ Problem statement + key requirements (functional & non-functional)
- □ High-Level Design (HLD) — components, data flow, architecture
- □ Low-Level Design (LLD) — classes, interfaces, DB schema
- □ Scalability + bottleneck discussion
- □ Possible follow-up / clarifying questions jo interviewer puch sakta hai
- □ Extra FAANG tips — edge cases, trade-offs, gotchas

Pro Tip □

Interview mein pehle 5 min clarifying questions pucho. Scope define karo. Assumptions state karo. Phir design start karo. Ye habit alone tumhe baaki candidates se differentiate karti hai.

□ System Design Framework (Every Question Pe Apply Karo)

Har system design answer mein ye 7-step framework follow karo:

1. Requirements Clarification — Functional kya chahiye? Non-functional (scale, latency, availability)?
2. Capacity Estimation — DAU, QPS, storage, bandwidth estimate karo.
3. High-Level Design — Core components draw karo: clients, LB, services, DB, cache, queue.
4. Data Model / Schema — Entities aur relationships define karo.
5. API Design — REST/gRPC endpoints define karo.
6. Deep Dive — Bottlenecks identify karo, scale karo, trade-offs discuss karo.
7. Wrap-Up — Monitoring, alerting, failure scenarios cover karo.

Trade-Off	When to Use
Consistency vs Availability	CAP theorem — partition tolerance given, choose C or A
SQL vs NoSQL	Structured + ACID = SQL; scale + flexibility = NoSQL
Sync vs Async	User-facing = sync; background jobs = async (Kafka/SQS)
Push vs Pull	Push = low latency; Pull = simpler, less infra pressure
Sharding Strategy	Hash-based (uniform), Range-based (range queries), Geographic

SD-01 · Reliable Webhook Dispatcher (Kafka-Powered)

[Kafka](#) | [Retry Logic](#) | [Idempotency](#) | [At-least-once delivery](#) | [Dead Letter Queue](#)

Problem Statement

Ek webhook dispatcher design karo jahan external services ko HTTP callbacks deliver ho — reliably, even if the target is temporarily down.

Requirements

Functional Requirements

- Users register webhook endpoints (URL, secret, event types)
- Events trigger HTTP POST to registered URLs
- Failed deliveries retry with exponential backoff
- Delivery logs + status dashboard available

Non-Functional Requirements

- At-least-once delivery guarantee
- High throughput: 100K+ events/sec
- P99 delivery latency < 5 seconds
- Fault tolerant: dispatcher crash should not lose events

High-Level Design

Event Producer → Kafka Topic → Dispatcher Workers → Target HTTP Endpoint

Component	Responsibility
API Gateway	Event ingestion, webhook CRUD registration
Kafka	Durable event queue, topic-per-event-type or single topic with routing key
Dispatcher Service	Kafka consumer pool, calls target URLs via HTTP
Retry Manager	Exponential backoff; pushes to retry-topic after failure
Dead Letter Queue	Permanently failed events after N retries
Webhook Registry DB	PostgreSQL — stores endpoint configs, secrets
Delivery Log DB	Cassandra / TimeSeries — stores per-event delivery status
Dashboard	React UI — shows delivery success/failure rates

Deep Dive: Reliability

- Idempotency Key: Har event ko unique ID milta hai; target server dedup kar sake
- At-least-once: Kafka offset commit AFTER successful HTTP 2xx response

- Exponential Backoff: 1s → 2s → 4s → 8s → 16s → DLQ
- Circuit Breaker: Agar target consistently fail kare, temporarily bypass karo
- HMAC Signature: Event payload sign karo with webhook secret for security

Data Schema

webhooks: `id, user_id, url, secret, events[], active, created_at`

delivery_logs: `event_id, webhook_id, attempt, status, response_code, ts`

Possible Follow-up Questions (Interviewer Puch Sakta Hai)

- How do you handle ordering guarantees? → Kafka partition by webhook_id
 - How to handle slow consumers? → Consumer lag monitoring, auto-scale workers
 - How to prevent replay attacks? → Timestamp in payload + HMAC validation
 - What if Kafka goes down? → Kafka itself replicates; use min ISR=2
 - How to test webhooks locally? → ngrok / webhook.site integration
-

SD-02 · Concurrent History Tracking System

[Event Sourcing](#) | [CQRS](#) | [Optimistic Locking](#) | [Audit Log](#) | [Change Data Capture](#)

Problem Statement

Registered entities (User, Product, Order) ke changes track karo — who changed what, when, and what was the previous value.

Core Design Patterns

Event Sourcing

State store mat rakho — sirf events store karo. Current state = replay of all events.

- ChangeEvent: { entity_type, entity_id, field, old_value, new_value, changed_by, timestamp }
- Entity ka current state = fold over all its events

CQRS Pattern

Command side: write changes + emit events. Query side: materialized view of current state.

- Write Path: API → Command Handler → Validate → DB Write + Emit Event to Kafka
- Read Path: Kafka Consumer → Build projection → Elasticsearch for full-text history search

Concurrency Handling

- Optimistic Locking: Entity mein version field. UPDATE WHERE version = expected_version. Fail = conflict.
- Idempotent Events: event_id UUID se dedup karo at consumer side
- Vector Clocks: Distributed nodes pe causal ordering guarantee (advanced)

Schema

entity_registry: entity_type, entity_id, schema_version

history_events: event_id UUID, entity_type, entity_id, version INT, patch JSONB, actor_id, ts TIMESTAMPTZ

Follow-up Questions

- How to rollback? → Replay events up to a given timestamp
 - How to diff two versions? → JSON patch between version N and N-1
 - Storage blowup? → Event compaction, snapshot every N events
-

SD-03 · Notification System (Multi-Channel)

[Kafka](#) | [Priority Queues](#) | [SMS/Email/Push](#) | [Rate Limiting](#) | [Template Engine](#)

Problem Statement

Ek scalable notification system design karo jo SMS, Email, Push Notifications aur In-App notifications support kare with high reliability.

Architecture

Component	Role
Notification API	Accept notification requests from internal services
Priority Queue	P0 (OTP/alerts) → P1 (transactional) → P2 (marketing)
Channel Router	Decide which channel(s) to use per user preferences
SMS Provider	Twilio / AWS SNS with fallback providers
Email Provider	SendGrid / AWS SES with retry
Push Provider	FCM (Android) / APNs (iOS)
In-App Store	Cassandra — stores unread notifications per user
Template Engine	Mustache/Handlebars — personalized message rendering
Preference Service	User's channel preferences + Do Not Disturb settings
Delivery Tracker	Per-notification status: queued → sent → delivered → failed

Scalability Tricks

- Batch emails: Group 1000 emails per SendGrid API call → 100x cost reduction
- Fan-out on write for push: Pre-compute device tokens list, parallel dispatch
- Rate limiting per user: 10 SMS/day, 50 emails/day to avoid spam
- Dead Letter Queue: Failed after 3 retries → DLQ → manual review

Follow-ups

- International SMS routing? → Country-based provider selection (cheaper local routes)
 - Notification storms? → Debounce — batch 10 similar events into 1 notification
 - How to A/B test templates? → Feature flags with experiment allocation
-

SD-04 · Auction Service

[Elasticsearch](#) | [Distributed Locks](#) | [WebSockets](#) | [Kafka](#) | [Redis](#)

Problem Statement

Ek auction platform design karo: Sellers products list karein, bidders search karein, aur highest bidder wins — with concurrent bid handling.

Core Flows

1. Seller Lists Product

- POST /auctions → creates auction with start_price, end_time, product details
- Product indexed in Elasticsearch for search
- Kafka event: auction.created → notification service

2. Bidder Searches

- GET /search?q=&category=&min_price=&max_price= → Elasticsearch query
- Fuzzy search + faceted filters + sorting by price/time_left

3. Placing a Bid (Critical Section)

Race condition hai: two bidders same time pe same auction pe bid lagayen.

Concurrency Solution: Redis Distributed Lock

- SETNX auction:{id}:lock {bidder_id} PX 500 → atomic lock with 500ms TTL
- Lock acquired → validate bid > current_highest → UPDATE DB → release lock
- Lock failed → retry with jitter or return 409 Conflict
- DB level: Optimistic lock via version column as second safety net

4. Winner Notification

- Auction end time → scheduled job checks auctions → determines winner
- Kafka event: auction.ended → Notification service sends email/SMS to seller + winner
- Payment service triggered for winner's payment

Data Model

auctions: id, seller_id, title, description, start_price, current_price, end_time, winner_id, status

bids: id, auction_id, bidder_id, amount, placed_at

Follow-ups

- Sniping prevention (last-second bids)? → Extend auction by 2 min if bid placed in last 2 min
 - Real-time bid updates? → WebSocket connection, Kafka consumer pushes updates
 - Fraud detection? → ML model on bid patterns, IP-based rate limiting
-

SD-05 · Cab Booking App (Ola / Uber)

[Geospatial](#) | [Kafka](#) | [Location Tracking](#) | [Matching Algorithm](#) | [Surge Pricing](#)

Key Services

Service	Responsibility
Location Service	Drivers send GPS every 5s → stored in Redis Geo
Matching Service	Find nearest available driver within radius
Booking Service	Create trip, assign driver, manage state machine
Pricing Service	Dynamic surge pricing based on supply/demand ratio
Trip Service	Track ongoing trip, ETA, route
Payment Service	Post-trip billing, wallet, UPI integration
Notification Service	Driver/rider updates via push + SMS
Rating Service	Post-trip rating for driver + rider

Driver-Rider Matching

- Redis GEORADIUS to find drivers within 2km
- Filter: available, rating > 3.5, car type matches
- Rank by: distance + ETA (Google Maps API)
- Send offer to top-3 drivers in parallel; first accept wins
- Timeout 10s → next batch of drivers offered

Surge Pricing Formula

$\text{surge_multiplier} = \text{demand_qps} / \text{supply_available_drivers}$

- If ratio > 2: 1.5x | ratio > 3: 2.0x | ratio > 5: 3.0x
- Recalculated every 30 seconds per geographic hexagon (H3 index)

Trip State Machine

REQUESTED → DRIVER_ACCEPTED → DRIVER_ARRIVED → TRIP_STARTED → TRIP_ENDED → PAYMENT_DONE

Follow-ups

- Location data at scale? → Kafka ingests 1M GPS pings/sec, Redis Geo for live query
- Driver goes offline mid-trip? → Kafka event → retry with new driver or SOS flow
- ETA accuracy? → Historical travel time ML model (traffic aware)
- Pool rides? → Route optimization (TSP variant), shared matching pool

SD-06 · Subscription Model (Razorpay / CureFit Style)

[Recurring Billing](#) | [Mandate](#) | [Retry Logic](#) | [Webhook](#) | [Idempotency](#)

Problem Statement

Monthly/yearly recurring payments design karo with mandate setup, auto-debit, retry on failure, aur confirmation notifications.

Flow

- Customer subscribes → Plan selected → Razorpay Mandate created (UPI AutoPay / NACH)
- Mandate approved by bank → subscription.active event fired
- Billing date aane pe → Cron job triggers charge attempt
- Payment success → Receipt email + unlock service access
- Payment failure → Retry T+1, T+3, T+7 → Cancel after 3 failures

Architecture Components

Component	Role
Subscription DB	PostgreSQL: plans, subscriptions, billing_cycles
Billing Engine	Cron job (Quartz/Celery Beat) triggers due bills
Payment Gateway	Razorpay API for charge + mandate management
Retry Service	Exponential retry with configurable schedule per plan
Idempotency Store	Redis: prevent double-charge with payment_key dedup
Webhook Handler	Razorpay webhooks → subscription state updates
Dunning Service	Failed payment emails with payment link (self-heal)

Idempotency Critical!

Double-Charge Prevention

- Har charge attempt ke liye idempotency_key = subscription_id + billing_cycle_date
- Redis SETNX se check karo before calling Razorpay API
- Payment gateway bhi idempotency key support karta hai (Razorpay X-idempotency-key header)

Follow-ups

- Plan upgrade mid-cycle? → Proration calculation (days remaining × price diff)
- Annual discount? → Create different plan_id with 2-month free offer
- Grace period? → 3-day grace after failed payment before service suspend

SD-07 · Trending Topic Service (Real-Time)

[Redis Sorted Sets](#) | [Flink/Spark Streaming](#) | [Sliding Window](#) | [Decay Function](#) | [Twitter HeavyHitters](#)

Problem Statement

Real-time trending topics identify karo (like Twitter trends) — based on recent spike in mentions, not just absolute count.

Key Insight

Trending ≠ Most Popular

'Cricket' har din popular hai. Trending = abnormal spike compared to baseline.

- Score = $\text{recent_count} / \text{historical_average} \times \text{time_decay_factor}$

Architecture

Component	Role
Event Ingestion	Kafka — ingests tweets/posts/searches at millions/sec
Stream Processor	Apache Flink — sliding window aggregation (5-min windows)
Heavy Hitters	Count-Min Sketch algorithm — approximate top-K topics (memory efficient)
Trend Scorer	Compares sliding window count vs 7-day rolling average
Redis Sorted Set	ZADD trends score topic — atomic top-K retrieval
Trend API	ZREVRANGE trends 0 9 → top 10 trends (< 1ms)
Personalization	Filter trends by user's language, location, interests

Time Decay

score = count × $e^{(-\lambda t)}$ where λ controls how fast old events lose weight

- $\lambda = 0.1$ → trends last ~10 hours
- $\lambda = 1.0$ → trends last ~1 hour (news cycle style)

Follow-ups

- Memory efficiency at scale? → Count-Min Sketch uses $O(\text{width} \times \text{depth})$ fixed memory
 - Geo-specific trends? → Partition Redis by region, separate Flink job per geo
 - Spam/botnet flooding? → Account quality score weights event contribution
-

SD-08 · Distributed Cache Service

[Consistent Hashing](#) | [LRU/LFU/TTL](#) | [Write-Through/Write-Back](#) | [Cache Stampede](#) | [Replication](#)

Eviction Policies

Policy	When to Use
LRU (Least Recently Used)	General purpose — good for temporal locality
LFU (Least Frequently Used)	Stable popular items better retained
TTL (Time-To-Live)	Auto-expire stale data — simplest to implement
SLRU (Segmented LRU)	Protected + probationary segments — better than LRU
ARC (Adaptive Replacement)	Self-tuning LRU+LFU hybrid — best hit rate

Write Strategies

- Write-Through: DB + Cache simultaneously. Consistent, but write latency doubles.
- Write-Back (Behind): Write to cache only, async flush to DB. Fast writes, data loss risk.
- Write-Around: Skip cache on write. Good for write-once data (logs, blobs).

Consistent Hashing

- Virtual nodes per server (150 vnodes) → uniform distribution even with few servers
- Add/remove server → only K/N keys remapped (vs all keys in mod-based hashing)

Cache Stampede (Thundering Herd) Prevention

Solutions

- Probabilistic early expiry: randomly re-compute before actual TTL expiry
- Lock-based: Redis SETNX lock → only one request recomputes, others wait
- Stale-while-revalidate: Serve stale, recompute in background

Follow-ups

- Cache invalidation across nodes? → Version-based invalidation or pub-sub invalidation
 - Hot key problem? → Local in-process cache layer (L1) in front of Redis (L2)
 - Persistence? → Redis RDB + AOF; separate cache cluster from persisted data
-

SD-09 · Authentication & Authorization Service

[JWT](#) | [OAuth 2.0](#) | [OTP](#) | [RBAC/ABAC](#) | [Rate Limiting](#)

Auth Flows Supported

Flow	Details
OTP Login	Phone → SMS OTP (Redis TTL 5min, max 3 attempts)
Email/Password	bcrypt hash, pepper + salt, credential stuffing protection
OAuth 2.0 / OIDC	Google / GitHub login, authorization code flow + PKCE
Magic Link	Email link with signed JWT (15 min expiry)
Refresh Token	Long-lived (7 days), stored in HttpOnly cookie

Token Strategy

- Access Token: JWT, 15-min expiry, signed with RS256 (asymmetric)
- Refresh Token: Opaque random token, stored in DB, rotate on use
- Token Revocation: Redis blacklist for logout (JWT is stateless by default)

Authorization Models

Model	Example	Use Case
RBAC	admin, editor, viewer roles	Simple apps, small orgs
ABAC	resource.owner == requester.id	Complex policies, multi-tenant
ReBAC (Google Zanzibar)	document:view#alice	Google Docs-like sharing

Security Hardening

- TOTP/HOTP for 2FA (Google Authenticator compatible)
- Device fingerprinting for suspicious login detection
- Adaptive risk scoring: new device + new location → step-up auth
- Brute force protection: exponential backoff per IP + account

Follow-ups

- SSO across microservices? → JWT verified at API gateway, services trust gateway
 - Token leak? → Short expiry + automatic token rotation on suspicious activity
 - Zero-trust? → Every internal service call also authenticated via mTLS + SPIFFE
-

SD-10 to SD-17 · Additional System Designs (Summary Format)

(Expanded questions — full depth available on request)

SD-10 · In-Memory Pub-Sub Model

Producers | **Consumers** | **Offsets** | **Consumer Groups** | **Backpressure**

- Topics → Partitions → Messages (offset-based)
- Consumer groups: each group gets all messages; within group, partitions split
- Offset management: committed offset = at-least-once; pre-fetch = at-most-once
- Backpressure: Consumer slow? → queue depth monitor → pause producer or scale consumer
- FAANG extra: How to implement compaction? → Keep only latest value per key

SD-11 · Config Manager Service

Versioning | **Hot Reload** | **Namespaces** | **ACLs** | **Audit Log**

- Zookeeper / etcd / Consul as backend for distributed config store
- Watch mechanism: clients long-poll or get pushed config changes via gRPC stream
- Namespaces: service_name/env/config_key hierarchy
- Blue-green config rollout with percentage rollout (feature flags style)
- FAANG extra: Config drift detection — alert if runtime != stored config

SD-12 · URL Shortener

Base62 | **Bloom Filter** | **Redis** | **Analytics** | **Geo-routing**

- ID generation: Snowflake ID → Base62 encode → 7-char short URL
- Collision check: Bloom Filter (fast negative lookup) before DB check
- Redirect: 301 (browser-cached, no analytics) vs 302 (always hits server, trackable)
- Analytics: Kafka event on each hit → ClickHouse for OLAP queries
- Custom URLs: Allow custom alias with uniqueness check + reserved words blocklist

SD-13 · Job Scheduler

Cron Parsing | **Distributed Lock** | **At-most-once vs At-least-once** | **Backfill**

- Quartz / Celery Beat style: Parse cron expression → next fire time
- Distributed lock before execution: Redis SETNX job:{id}:lock
- Missed job (server was down)? → Backfill policy: run once or skip
- Job types: one-time, recurring, event-triggered (dag-based)
- FAANG extra: Priority queue for jobs — critical infra jobs get workers first

SD-14 · Leaderboard System (LeetCode Style)

Redis Sorted Sets | **Partitioned Ranking** | **Global vs Contest** | **Tie Breaking**

- Redis ZADD leaderboard score user_id — $O(\log N)$ insert
- ZREVRANK user_id → $O(\log N)$ rank fetch
- Global leaderboard sharding: top 1000 in Redis, rest in DB with periodic sync
- Tie-breaking: Secondary sort by time_achieved ASC (faster solver wins)

- Contest leaderboard: Separate Redis key per contest, expire after contest end

SD-15 · Car Parking Application

[Real-time Availability](#) | [Floor Map](#) | [Reservation](#) | [License Plate Recognition](#)

- Parking spots → FloorMap → Zone → Spot (hierarchical model)
- Available spots: Redis bitmap per floor (spot_index → 0/1)
- Reservation: 15-min hold with Redis TTL, confirmed on payment
- Entry/Exit: LPR (License Plate Recognition) → auto match to reservation
- Overflow handling: Waitlist queue (Redis List), notify on cancellation

SD-16 · Trending Topic Service (Advanced Recap)

(See SD-07 for full design. Key FAANG additions:)

- Botnet detection: velocity check (same IP > 100 events/min = suspect)
- Multi-language trending: separate count-min sketch per language
- Trend prediction: ML model trained on historical spike patterns

SD-17 · Cache Service (Advanced Recap)

(See SD-08 for full design. FAANG extras:)

- Multi-tier cache: L1 (in-process) → L2 (Redis cluster) → L3 (DB)
 - Cache warming on startup: pre-load hot keys from recent access log
 - Cost optimization: tiered eviction — hot data in RAM, warm data in SSD-based Redis
-

□ Machine Coding Problems

Machine coding = LLD + implementation. 60-90 min mein working code likhna hai. Focus: clean design, SOLID principles, extensibility.

Machine Coding Formula

- 1. Read problem completely first (5 min)
 - 2. Identify entities + relationships → Draw class diagram mentally
 - 3. Define interfaces before implementations
 - 4. Write code iteratively: core flow first, edge cases later
 - 5. Test with provided examples + 2-3 edge cases yourself
-

MC-01 · Google Docs Mini

[Piece Table / Gap Buffer](#) | [Command Pattern](#) | [Navigation](#) | [Undo/Redo](#)

Requirements

- `append(text)`: Document ke end pe text add karo
- `replace(start, end, text)`: Character range replace karo
- `navigate(direction)`: Arrow keys, PgUp/PgDn cursor movement
- `display()`: Current document state show karo

Key Design Decisions

Data Structure	Trade-off
Naive String	$O(N)$ for every insert — terrible for large docs
Gap Buffer	$O(1)$ amortized near cursor — used in Emacs
Piece Table	$O(1)$ insert anywhere, immutable original — used in VSCode
Rope	$O(\log N)$ all ops — used for very large documents

Core Classes

Document: `content: PieceTable, cursor: int, history: Stack<Command>`

Command: `interface: execute(), undo() → AppendCommand, ReplaceCommand`

PieceTable: `original: str, added: str, pieces: List<Piece>`

Piece: `start: int, length: int, source: ORIGINAL | ADDED`

Navigation Logic

- Arrow LEFT/RIGHT: cursor ± 1 with bounds check
- Arrow UP/DOWN: Find current line, move to same column in prev/next line
- PgUp/PgDn: Move cursor by PAGE_SIZE lines (default 20)
- Home/End: Move to line start (find prev `\n`) / line end (find next `\n`)

Undo/Redo (Command Pattern)

- Execute command → push to `undo_stack`
- Undo → pop from `undo_stack` → push to `redo_stack`
- New command after undo → clear `redo_stack`

FAANG Follow-ups

- Collaborative editing? → Operational Transformation (OT) or CRDT
- Syntax highlighting? → Token-based parsing on background thread
- Search (Ctrl+F)? → Boyer-Moore or KMP for fast pattern matching

MC-02 · In-Memory Search Engine

[Inverted Index](#) | [TF-IDF](#) | [Namespaces](#) | [Pagination](#) | [Relevance Scoring](#)

Requirements

- `insert(namespace, doc_id, content)`: Document add karo
- `delete(namespace, doc_id)`: Document remove karo
- `search(namespace, query, sort)`: Keyword search with relevance sorting
- `search_by_tags(namespace, tags[])`: Tag-based filtering

Core Data Structure: Inverted Index

`word` → `{ doc_id: [positions] }` → fast document lookup by term

InvertedIndex: `Map<namespace, Map<term, Map<doc_id, List<position>>>>`

DocumentStore: `Map<namespace, Map<doc_id, Document>>`

TagIndex: `Map<namespace, Map<tag, Set<doc_id>>>`

Relevance Scoring (TF-IDF)

- $TF = \text{term_count_in_doc} / \text{total_words_in_doc}$
- $IDF = \log(\text{total_docs} / \text{docs_containing_term})$
- $\text{score} = TF \times IDF \rightarrow \text{higher} = \text{more relevant}$
- Sort results by score DESC for relevance-ranked results

FAANG Follow-ups

- Fuzzy search? → Edit distance (Levenshtein) or BK-Tree
 - Autocomplete? → Trie on indexed terms with prefix search
 - Phrase search ('hello world' exact)? → Position checking in inverted index
 - Large dataset? → Segment files + merge (LSM-tree approach, like Lucene)
-

MC-03 · Survey Management System

[Weighted Scoring](#) | [Concurrency Safe Submission](#) | [ReentrantLock](#) | [Analytics](#)

Requirements

- Admin: create survey (title, questions), set question weights
- Users: submit responses (concurrent submissions must be safe)
- Fetch average ratings per question + overall survey score
- Prevent duplicate submissions from same user

Key Classes

Survey: `id, title, questions: List<Question>, status: ACTIVE|CLOSED`

Question: `id, text, type: RATING|MCQ|TEXT, weight: double, options: List<String>`

Submission: `id, survey_id, user_id, answers: Map<question_id, answer>, submitted_at`

SurveyService: `createSurvey(), submitResponse(), getAverageRatings()`

Concurrency Handling

Thread-Safe Submission

- `ConcurrentHashMap` for submission store (thread-safe)
- `ReentrantLock` per `survey_id` for duplicate check + insert (atomic)
- `AtomicLong` for ID generation
- Synchronized method for rating accumulation or `AtomicReference<Double>`

Weighted Score Calculation

$$\text{overall_score} = \frac{\sum (\text{question_avg} \times \text{question_weight})}{\sum (\text{question_weights})}$$

FAANG Follow-ups

- Survey branching (skip logic)? → Conditional question visibility based on prev answer
 - Anonymity? → Strip `user_id` before storing, keep only aggregate
 - Real-time results? → `WebSocket` push on each submission to admin dashboard
-

MC-04 · In-Memory SQL Database

[B-Tree Index](#) | [Parser](#) | [Query Executor](#) | [Transactions](#) | [ACID](#)

Supported Operations

- CREATE TABLE, DROP TABLE
- INSERT INTO, UPDATE, DELETE FROM
- SELECT with WHERE, ORDER BY, LIMIT, basic JOINS
- BEGIN TRANSACTION, COMMIT, ROLLBACK

Architecture

Component	Role
SQL Parser	Tokenize + parse SQL string into AST
Query Planner	Choose execution strategy (full scan vs index)
Executor	Execute AST against in-memory tables
Table Store	Map<table_name, List<Map<col, value>>>
Index Store	Map<table_name, Map<col, TreeMap<value, Set<row_id>>>>
Transaction Manager	MVCC or lock-based for concurrent transactions

Index Implementation

- Primary key: HashMap for O(1) lookup
- Secondary index: TreeMap (sorted) for range queries (BETWEEN, >, <)
- Full scan: iterate all rows — only when no index available

FAANG Follow-ups

- MVCC? → Each row has version chain; readers see snapshot at transaction start
 - Aggregate functions (SUM, COUNT, AVG)? → Post-filter reduce operation
 - Foreign keys? → Reference integrity check on insert/delete
-

MC-05 · Version Control System (Git-like)

[DAG](#) | [SHA hashing](#) | [Branching](#) | [Merge](#) | [Diff](#)

Core Concepts

Object	Description
Blob	File content stored as SHA-256 hash of content
Tree	Directory snapshot: Map<filename, blob_sha>
Commit	{ tree_sha, parent_sha, message, author, timestamp }
Branch	Named pointer to a commit SHA
HEAD	Current active branch/commit

Key Operations

init(): Create .vcs/ directory with empty HEAD

add(file): Compute blob hash, stage in index

commit(msg): Create tree from index, create commit, advance HEAD

branch(name): Create new ref pointing to HEAD commit

checkout(branch): Update HEAD, restore working tree to branch's commit

log(): Walk parent chain from HEAD, print commit history

diff(c1, c2): Compare trees of two commits, show changed files

Merge Strategy

- Find LCA (Lowest Common Ancestor) of two branch tips → 3-way merge
- No conflict: auto-merge. Conflict: mark <<<<<< / >>>>>> markers
- Fast-forward merge: if one branch is direct ancestor of other

FAANG Follow-ups

- Rebase? → Replay commits from branch onto new base one by one
 - Stash? → Stack of work-in-progress diffs, pop on stash apply
 - Pack objects? → Delta compress similar blobs to save space
-

MC-06 · In-Memory Splitwise

[Expense Splitting](#) | [Balance Simplification](#) | [Graph Reduction](#) | [Settle-up](#)

Requirements

- Users add expenses: who paid, total amount, split among whom (equal/exact/percent)
- View balances: who owes whom how much
- Settle-up: simplify balances (minimize transactions needed)

Core Classes

User: `id, name, email`

Group: `id, name, members: List<User>, expenses: List<Expense>`

Expense: `id, paid_by, total, splits: List<Split>, category, description`

Split: `user_id, amount` (user owes this much for this expense)

Balance Calculation

- For each expense: `paid_by` gets credited total; each member debited their split amount
- Net balance per user pair: $\text{net}[A][B] = \text{amount A owes B}$ (negative = B owes A)

Balance Simplification Algorithm

Greedy Minimization

- Compute net balance per user (positive = creditor, negative = debtor)
- MaxHeap for creditors, MinHeap for debtors
- Greedily match: max creditor gets paid by max debtor
- Result: $\leq N-1$ transactions to settle N people

Split Types

- Equal split: total / N per person
- Exact split: explicitly specify each person's amount (must sum to total)
- Percentage split: specify % per person (must sum to 100)

FAANG Follow-ups

- Currency conversion? → Store in base currency, apply conversion on display
 - Recurring expenses (rent)? → Scheduled expense creation each month
 - Expense history? → Event log per group, queryable by date/category
-

MC-07 · Persistent Pub-Sub Queue

[Ordered Delivery](#) | [Offset Tracking](#) | [Consumer Groups](#) | [At-least-once](#)

Design

Topic: `name, partitions: List<Partition>`

Partition: `id, messages: List<Message> (append-only log)`

Message: `offset: long, key, value, timestamp`

ConsumerGroup: `id, topic, committed_offsets: Map<partition, offset>`

Key Behaviors

- Producers write to partition ($\text{hash}(\text{key}) \% \text{num_partitions}$ for ordering)
 - Consumers pull from partition starting at `committed_offset`
 - Commit offset after processing → at-least-once guarantee
 - Multiple consumer groups → each gets independent offset tracking
 - Persistence: WAL (Write-Ahead Log) to disk with `fsync` for durability
-

MC-08 · Reusable Dropdown Component

[Virtual Scrolling](#) | [Debounced Search](#) | [Keyboard Nav](#) | [Accessibility](#)

Requirements

- Large dataset (50K+ options) render karna without lag
- Search with debounce (300ms)
- Keyboard navigation (Up/Down/Enter/Esc)
- Multi-select support
- ARIA accessible

Virtual Scrolling

Key Insight

DOM mein sirf visible items render karo (~10-15 items). Baaki ke liye spacer divs.

- $\text{container_height} = \text{total_items} \times \text{item_height}$
- $\text{visible_start} = \text{Math.floor}(\text{scrollTop} / \text{item_height})$
- $\text{visible_end} = \text{visible_start} + \text{visible_count}$
- Render only items[visible_start..visible_end]

FAANG Follow-ups

- Grouped options (optgroup)? → Flatten with type: 'group_header' | 'option'
 - Async loading? → Trigger API call on search, show loading spinner
 - Infinite scroll? → Load next page when scroll reaches 80% of container
-

MC-09 · Search Engine with Tags

[Inverted Index](#) | [Tag Filtering](#) | [Relevance + Recency](#) | [Boolean Queries](#)

Requirements

- `add_document(id, content, tags[])`
- `remove_document(id)`
- `search(keywords, tags, sort_by)`: Combined keyword + tag search
- Boolean: AND (all keywords must match), OR (any), NOT (exclude)

Data Structures

inverted_index: `Map<word, Set<doc_id>>`

tag_index: `Map<tag, Set<doc_id>>`

doc_store: `Map<doc_id, { content, tags, timestamp, tfidf_scores }>`

Query Processing

- Tokenize + stem query terms (remove stop words, lowercase, stem)
 - AND: intersect all posting lists → docs containing ALL terms
 - OR: union all posting lists → docs containing ANY term
 - NOT: subtract posting list from result set
 - Tag filter: `tag_index[tag]` → intersect with keyword results
-

□ Extra FAANG Questions (Jo Tumse Puch Sakte Hain)

Extra System Design Questions

Question	Key Concepts
Design Google Docs (Collaborative)	OT / CRDT, WebSocket sessions, operational transform merge
Design WhatsApp	Message queue per user, E2E encryption, offline delivery, receipts
Design Instagram Feed	Fan-out on write (celebrities) vs fan-out on read, pre-computed feeds
Design Dropbox/Google Drive	Chunking, delta sync, version control, conflict resolution
Design a Rate Limiter	Token bucket, leaky bucket, sliding window log, Redis Lua script
Design Search Autocomplete	Trie with DFS, Elasticsearch prefix query, caching top suggestions
Design a Payment System	2-phase commit, idempotency, saga pattern, PCI compliance
Design YouTube/Netflix	CDN, adaptive bitrate streaming (HLS/DASH), video encoding pipeline
Design Twitter Timeline	Push model for regular users, pull for celebrities, hybrid approach
Design a Recommendation Engine	Collaborative filtering, matrix factorization, feature store

Extra Machine Coding Questions

Question	Key Concepts
Design a Chess Game	Board representation, piece movement validation, check/checkmate detection
Design a Parking Lot	OOP: Spot hierarchy, vehicle types, ticket system, payment
Design an Elevator System	State machine, scheduling (LOOK algorithm), multi-elevator coordination
Design a Food Delivery App (LLD)	Restaurant, Menu, Order, Delivery Partner, Rating entities
Design a Library Management System	Books, Members, Borrowing, Fines, Reservations
Design a Movie Ticket Booking	Theaters, Shows, Seats, Reservations (concurrent), Payment
Design a Hotel Booking System	Rooms, Availability calendar, Pricing rules,

Question	Key Concepts
	Overbooking strategy
Design a Log Aggregation System	Structured logs, in-memory circular buffer, flush to disk on threshold
Design a Task Management (Jira-like)	Tasks, Sprints, Boards, Transitions, Assignments
Design a Trie-based Phone Directory	Search by name/number, autocomplete, contacts import

□ Quick Reference Cheat Sheet

Technology Selection Guide

Need	Technology	Use Case
Relational Data + ACID	PostgreSQL / MySQL	User accounts, payments, orders
Wide-column / High write	Cassandra / DynamoDB	Event logs, IoT, time-series
Document Store	MongoDB	Flexible schema, product catalogs
Cache	Redis	Session, hot data, distributed locks
Search	Elasticsearch	Full-text, faceted, fuzzy search
Message Queue	Kafka	High-throughput event streaming
Task Queue	RabbitMQ / SQS	Job distribution, work queues
File Storage	S3 / GCS	Blobs, images, video
CDN	CloudFront / Fastly	Static assets, global edge delivery
Service Mesh	Istio / Envoy	mTLS, traffic control, observability

Capacity Estimation Quick Reference

Metric	Rule of Thumb
1M DAU, 10 req/day	~116 QPS average, ~350 QPS peak
1KB per request	$1M \times 1KB = 1GB/day$ storage
Twitter-scale	500M tweets/day = ~5K writes/sec
Latency budget (web)	Total 200ms: DB 50ms, Cache 1ms, Network 20ms
99.9% SLA	8.7 hours downtime/year
99.99% SLA	52 minutes downtime/year

SOLID Principles (Machine Coding)

Principle	Hinglish Explanation
S — Single Responsibility	Ek class, ek reason to change. PaymentProcessor $\neq$ EmailSender
O — Open/Closed	Open for extension, closed for modification → use interfaces

Principle	Hinglish Explanation
L — Liskov Substitution	Subclass substitutable for parent without breaking behavior
I — Interface Segregation	Thin interfaces > fat interfaces. ISP saves implementors
D — Dependency Inversion	Depend on abstractions, not concretions → Dependency Injection

Common Design Patterns (Machine Coding)

Pattern	Use When	Example
Strategy	Runtime algorithm swap	SortStrategy: BubbleSort, QuickSort
Observer	Event notification	EventBus, Kafka consumer analogy
Command	Encapsulate actions	Undo/Redo in text editor
Factory	Object creation	NotificationFactory → SMSNotification EmailNotification
Singleton	One instance globally	ConfigManager, ConnectionPool
Decorator	Dynamic feature add	LoggingCache wraps BaseCache
Builder	Complex object construction	QueryBuilder, DocumentBuilder
State	State machine	Order: PLACED→SHIPPED→DELIVERED

☐ Interview Tips (Jo Topper Karte Hain)

System Design Interview Tips

13. Start with clarifying questions: 'Kitne users? Read-heavy ya write-heavy? Global ya India-only?'
14. Estimate before designing: 'QPS ~1000, storage ~10TB/year, so we need distributed setup'
15. Draw while talking: components, arrows (sync/async), DB, cache — verbalize everything
16. Mention trade-offs proactively: 'SQL use kar sakte hain lekin at this scale NoSQL better rahega kyunki...'
17. Handle interviewer hints gracefully: 'Good point — let me reconsider the consistency model here'
18. End with: monitoring (Prometheus/Grafana), alerts, SLAs, failure scenarios

Machine Coding Interview Tips

19. Read problem fully, don't start coding immediately — identify all entities first
20. Define interfaces before writing implementations
21. Write TODO comments for non-critical parts, implement core first
22. Test with provided examples as you code — don't wait till the end
23. Talk about what you would add with more time: 'Persistence, rate limiting, etc.'

Common Mistakes to Avoid

- ☐ Jumping to solution without requirements clarification
- ☐ Over-engineering: 'We need ML model for...' when simple heuristic works
- ☐ Ignoring failure scenarios (servers crash, network partition, DB goes down)
- ☐ Forgetting security: auth, rate limiting, input validation
- ☐ Perfect code over working code in machine coding
- ☐ Staying silent — always narrate your thinking process

Final Reminder ☐

Interviewer ko tumhara thought process dekhna hai, not a perfect answer. Communication + structured thinking > perfect design.

Practice 1 system design + 1 machine coding per day for 30 days = FAANG ready! ☐